

PANDAS

Complete Study Notes

Data Manipulation & Analysis with Python

Python · Data Science · Machine Learning

Reference Guide — 2024 Edition

A concise, structured reference for working with Pandas — covering core data structures, cleaning, aggregation, merging, time series, and performance essentials.

Table of Contents

1.	Introduction to Pandas	3
2.	Series and DataFrames	3
3.	Reading & Writing Data	4
4.	Data Selection & Filtering	5
5.	Data Cleaning	6
6.	Sorting & Basic Statistics	7
7.	Grouping & Aggregation	8
8.	Merging & Joining	9
9.	Time Series Operations	10
10.	Advanced Tricks & Performance	11
11.	Handling Large Datasets	12

1 · Introduction to Pandas

Pandas is an open-source Python library built on top of NumPy that provides fast, flexible, and expressive data structures designed to make working with structured (tabular, multidimensional, potentially heterogeneous) and time-series data both easy and intuitive. It is one of the most widely used tools in data science and machine learning pipelines.

Why Pandas?

- **DataFrame abstraction** — think of it as an in-memory spreadsheet with labelled rows and columns.
- **Missing-data handling** — built-in NaN detection, filling, and dropping.
- **Time-series support** — resampling, rolling windows, date parsing.
- **Ecosystem integration** — seamlessly interoperates with NumPy, Matplotlib, and Scikit-learn.
- **File I/O** — read/write CSV, Excel, JSON, SQL, Parquet, and more.

Install: `pip install pandas` | Import convention: `import pandas as pd`

Core Data Structures

Pandas exposes two primary data structures. A **Series** is a one-dimensional labeled array capable of holding any data type — integers, floats, strings, or Python objects. A **DataFrame** is a two-dimensional, size-mutable, tabular data structure with labeled axes (rows and columns). Each column of a DataFrame is effectively a Series.

2 · Series and DataFrames

Series

A Series is a one-dimensional labeled array. Labels form the index. When created from a plain list the index defaults to 0, 1, 2 ... A custom index can be supplied explicitly, or inferred from a dictionary.

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

print(s['b']) # 20

# From dictionary
s = pd.Series({'Alice': 25, 'Bob': 30, 'Charlie': 35})
```

DataFrame

A DataFrame is a two-dimensional labeled data structure where each column can hold a different data type. It can be constructed from dictionaries, lists of lists, NumPy arrays, or loaded directly from files.

```
data = {  
  
    'Name': ['Alice', 'Bob', 'Charlie'],  
  
    'Age': [25, 30, 35],  
  
    'Salary': [50000, 60000, 70000],  
  
}  
  
df = pd.DataFrame(data)
```

Key Properties

df.shape	Tuple (rows, columns)
df.dtypes	Data type of each column
df.columns	Column name index
df.index	Row label index
df.describe()	Descriptive statistics for numeric columns
df.info()	Memory usage, dtypes, non-null counts

3 · Reading & Writing Data

Pandas provides a rich family of **read_*** and **to_*** functions for importing and exporting data in virtually every common format.

Reading Data

```
# CSV

df = pd.read_csv('data.csv', index_col=0, na_values=['NA'])

# Excel

df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

# JSON

df = pd.read_json('data.json')

# SQL

import sqlite3

conn = sqlite3.connect('db.sqlite')

df = pd.read_sql_query('SELECT * FROM employees', conn)
```

Writing Data

```
df.to_csv('output.csv', index=False)

df.to_excel('output.xlsx', sheet_name='Results', index=False)

df.to_json('output.json', orient='records')
```

*Always use **index=False** when writing CSVs unless the index itself is meaningful data.*

4 · Data Selection & Filtering

Pandas offers several ways to access data. Choosing the right accessor is important for both correctness and performance.

Column Selection

```
df['Age'] # Single column -> Series

df[['Name', 'Age']] # Multiple columns -> DataFrame
```

loc vs iloc

loc selects by label (row/column names). **iloc** selects by integer position (0-based).

```
df.loc[0, 'Age'] # Row index 0, column 'Age'

df.loc[0:2, 'Name':'City'] # Inclusive label slice

df.iloc[0, 1] # Row 0, column at position 1

df.iloc[0:3, 1:3] # Rows 0-2, columns 1-2 (exclusive end)
```

Boolean Filtering

```
young = df[df['Age'] < 30]

# Multiple conditions (wrap each in parentheses)

result = df[(df['Age'] > 25) & (df['Salary'] > 55000)]

# query() method

result = df.query('Age > 25 and Salary > 55000')

# isin() for membership testing

df[df['City'].isin(['New York', 'London'])]
```

5 · Data Cleaning

Real-world data is rarely clean. Pandas provides a comprehensive toolkit for detecting and handling missing values, duplicates, wrong types, and inconsistent strings.

Missing Values

```
df.isnull().sum() # Count NaN per column

df.dropna() # Drop rows with ANY NaN

df.dropna(subset=['Age']) # Drop only if Age is NaN

df.dropna(how='all') # Drop rows where ALL values are NaN

df.fillna(0) # Fill all NaN with 0

df['Age'].fillna(df['Age'].mean()) # Fill with column mean

df.fillna(method='ffill') # Forward-fill (propagate last valid)
```

Duplicates

```
df.duplicated().sum() # Count duplicate rows

df.drop_duplicates() # Remove duplicates (keep first)

df.drop_duplicates(subset=['Name']) # Based on subset of columns
```

Type Conversion

```
df['Age'] = df['Age'].astype(int)

df['Date'] = pd.to_datetime(df['Date'])

df['City'] = df['City'].astype('category') # Saves memory

# Coerce bad values to NaN instead of raising an error

df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
```

String Operations (.str accessor)

```
df['Name'].str.lower() # Lowercase

df['Name'].str.strip() # Remove whitespace

df['Name'].str.replace(' ', '_') # Replace character
```

```
df['Name'].str.contains('Alice') # Boolean mask
```

```
df['Name'].str.split(' ', expand=True) # Split into columns
```


6 · Sorting & Basic Statistics

Sorting

```
df.sort_values('Age') # Ascending

df.sort_values('Age', ascending=False) # Descending

df.sort_values(['City', 'Age'], # Multi-column
ascending=[True, False])

df.sort_index() # Sort by row index
```

Descriptive Statistics

mean() / median()	Arithmetic average / middle value
std() / var()	Standard deviation / variance
min() / max()	Minimum and maximum values
quantile(q)	q-th percentile (e.g. 0.25 for Q1)
describe()	Full summary: count, mean, std, min, quartiles, max
value_counts()	Frequency count of unique values
corr()	Pairwise correlation matrix (Pearson default)

Correlation values range from -1 (perfect negative) to +1 (perfect positive). A value near 0 implies no linear relationship between the two variables.

7 · Grouping & Aggregation

The **GroupBy** mechanism follows a split → apply → combine pattern: split the data into groups by one or more keys, apply an aggregation function to each group, then combine the results into a new DataFrame.

```
grouped = df.groupby('City')

# Single aggregation
grouped['Salary'].mean()

# Multiple aggregations via dict
grouped.agg({'Age': 'mean', 'Salary': ['sum', 'count']})

# Custom function
grouped['Age'].agg(lambda x: x.max() - x.min()) # range
```

Pivot Tables

Pivot tables provide a spreadsheet-style summary of data by aggregating values across two dimensions.

```
pd.pivot_table(df, values='Salary', index='City',
               aggfunc='mean', margins=True)
```

8 · Merging & Joining

Pandas supports SQL-style joins via **pd.merge()** and stacking via **pd.concat()**. Understanding join types is essential for combining data from multiple sources.

SQL-style Joins

inner	Keep only rows whose key exists in BOTH tables.
left	Keep all rows from the left table; NaN where no right match.
right	Keep all rows from the right table; NaN where no left match.
outer	Keep ALL rows from both tables; NaN where no match exists.

```
pd.merge(df1, df2, on='ID', how='inner') # inner join
```

```
pd.merge(df1, df2, on='ID', how='left') # left join
```

```
pd.merge(df1, df2, on='ID', how='outer') # outer join
```

Concatenation

```
pd.concat([df1, df2], axis=0) # Stack rows (vertical)
```

```
pd.concat([df1, df2], axis=1) # Stack columns (horizontal)
```

```
pd.concat([df1, df2], keys=['train', 'test']) # Multi-index
```

9 · Time Series Operations

Pandas has first-class support for date/time data. Once a column is parsed as **datetime64** and set as the index, powerful resampling and windowing operations become available.

Parsing and Indexing

```
df['Date'] = pd.to_datetime(df['Date']) # Parse strings

df = df.set_index('Date') # Use as index

pd.date_range('2024-01-01', periods=12, freq='M') # Date range
```

Resampling

Resampling changes the frequency of the time series — downsampling (e.g. daily to monthly) or upsampling (e.g. monthly to daily).

```
df.resample('D').mean() # Daily

df.resample('W').sum() # Weekly totals

df.resample('M').agg({'Sales': 'sum', 'Revenue': 'mean'})
```

Rolling & Expanding Windows

```
df['MA_7'] = df['Value'].rolling(7).mean() # 7-day moving avg

df['CumSum'] = df['Value'].expanding().sum() # Cumulative sum

df['Lag1'] = df['Value'].shift(1) # Lag by 1 period

df['PctChg'] = df['Value'].pct_change() # % change
```

10 · Advanced Tricks & Performance

apply() and **map()**

apply() calls a function along an axis of a DataFrame or on a Series. **map()** is Series-only and optimised for element-wise dictionary or function lookups.

```
# Column-wise apply

df.apply(lambda col: col.max() - col.min())

# Row-wise apply (axis=1)

df.apply(lambda row: row['Age'] + row['Salary'], axis=1)

# Custom function on a column

def grade(score):

    return 'A' if score >= 90 else 'B' if score >= 80 else 'C'

df['Grade'] = df['Score'].apply(grade)

# map() with dictionary

df['City_Code'] = df['City'].map({'New York': 'NY', 'London': 'LON'})
```

Performance Tips

- **Vectorised operations** are orders of magnitude faster than `apply()` for simple math.
- **Use category dtype** for low-cardinality string columns — reduces memory and speeds up groupby.
- **Avoid iterrows()**; prefer vectorised operations or `apply()` for complex row logic.
- **Read only needed columns** via `usecols=` in `read_csv` to reduce memory.
- **Chunked reading**: use `chunksize=` parameter for files too large to fit in RAM.

11 · Handling Large Datasets

Pandas is single-threaded and loads data entirely into RAM. For datasets exceeding a few gigabytes, consider purpose-built alternatives.

Library	Speed	Memory	Best For
Pandas	Baseline	Full load	Less than 1 GB
Polars	10-100x	Efficient	1-10 GB, speed priority
Dask	2-10x	Chunked	More than 10 GB, distributed

Polars Quick Example

```
import polars as pl

df = pl.read_csv('large.csv')

result = (df
    .filter(pl.col('age') > 30)
    .group_by('category')
    .agg(pl.col('value').mean())
)
```

Dask Quick Example

```
import dask.dataframe as dd

df = dd.read_csv('huge_*.csv') # Lazy, chunked

result = df.groupby('category')['value'].mean().compute()
```

Start with Pandas, profile your bottlenecks, then migrate to Polars for speed or Dask for out-of-memory workloads.